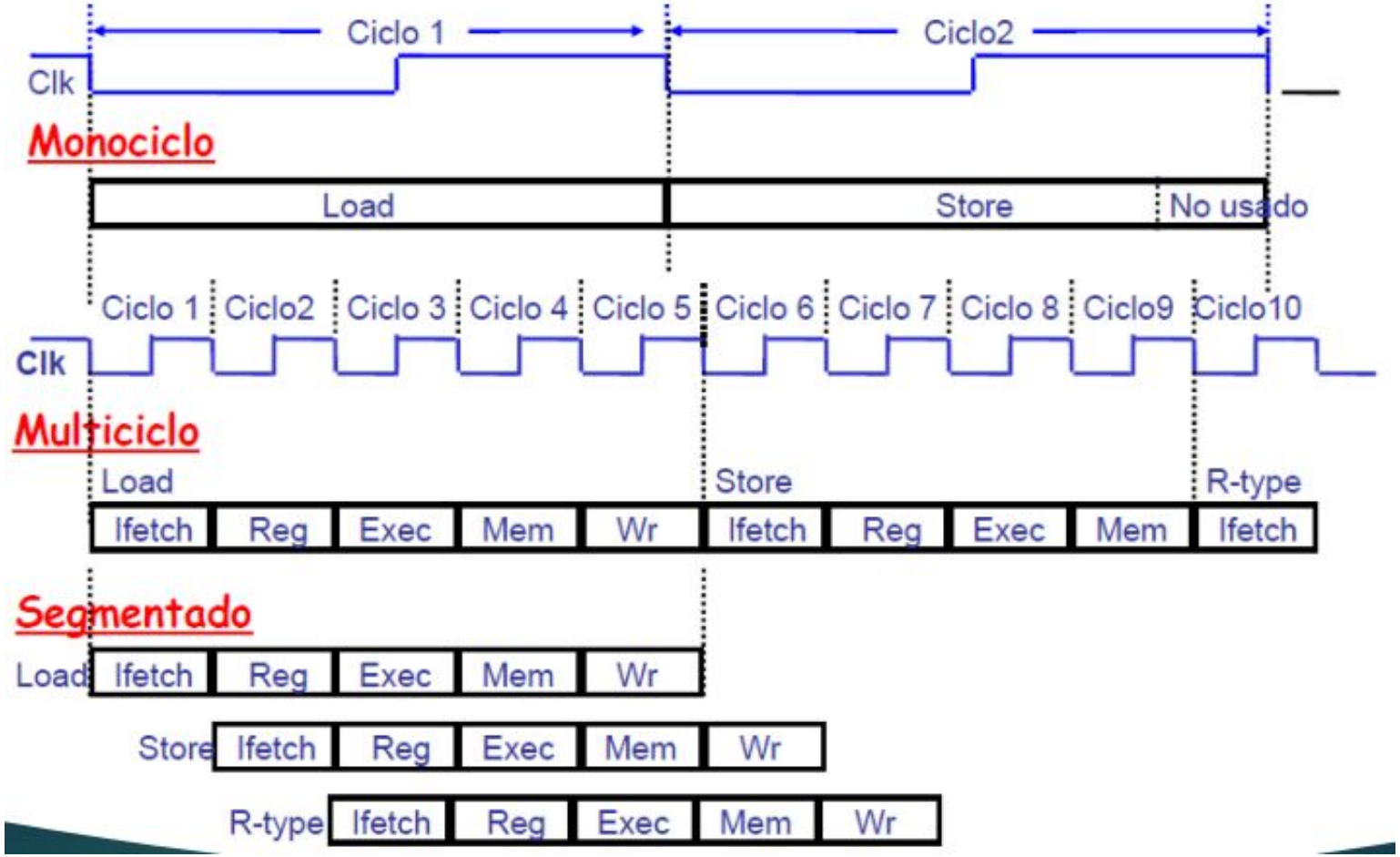
TRABAJO FINAL: PIPELINE PROCESADOR (RISC-V)

Consigna

- Implementar el **pipeline** del procesador **RISC-V**
- Implementar una **Debug Unit** que permite enviar y recibir información al procesador mediante protocolo **UART**.
- Una interfaz para visualizar los datos e interactuar con la **Debug Unit** (**CLI**, **GUI** y/o **TUI**).

Marco Teorico



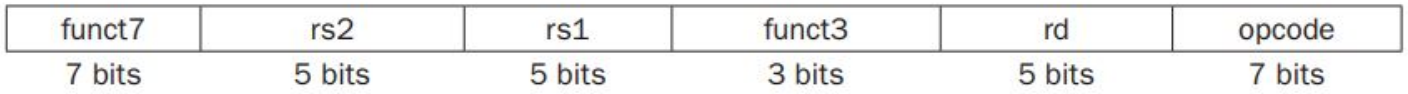
Etapas

- IF (Instruction Fetch): Búsqueda de la instrucción en la memoria de programa.
- ID (Instruction Decode): Decodificación de la instrucción y lectura de registros.
- EX (Execute): Ejecución de la instrucción propiamente dicha.
- MEM (Memory Access): Lectura o escritura desde/hacia la memoria de datos.
- WB (Write back): Escritura de resultados en los registros.

Riegos

- Tipos:
  - Estructurales: Se producen cuando dos instrucciones tratan de utilizar el mismo recurso en el mismo ciclo.
  - De datos: Se intenta utilizar un dato antes de que esté preparado. Mantenimiento del orden estricto de lecturas y escrituras.
  - De control: Intentar tomar una decisión sobre una condición todavía no evaluada.

Campos de las instrucciones



Here is the meaning of each name of the fields in RISC-V instructions:

- ***opcode***: Basic operation of the instruction, and this abbreviation is its traditional name.
- *rd:* The register destination operand. It gets the result of the operation.
- *funct3:* An additional opcode field.
- *rs1:* The first register source operand.
- *rs2:* The second register source operand.
- *funct7:* An additional opcode field.

Tipos de Instucciones

Tipo R

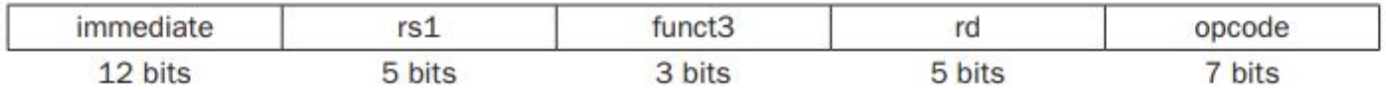
- Son operaciones aritméticas y lógicas

Instruction	Format	funct7	rs2	rs1	funct3	rd	opcode
add (add)	R	0000000	reg	reg	000	reg	0110011
sub (sub)	R	0100000	reg	reg	000	reg	0110011

Tipos de Instucciones

Tipo I

- Son operaciones Imediatas
- También operaciones de Load



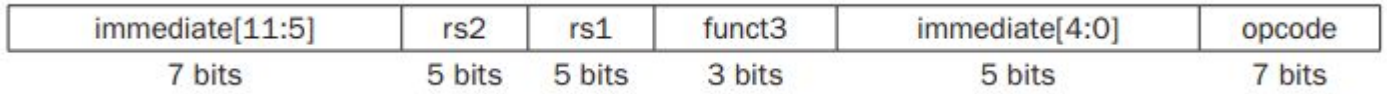
immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

Instruction	Format	immediate	rs1	funct3	rd	opcode
`addi` (add immediate)	I	constant	reg	000	reg	0010011
`ld` (load doubleword)	I	address	reg	011	reg	0000011

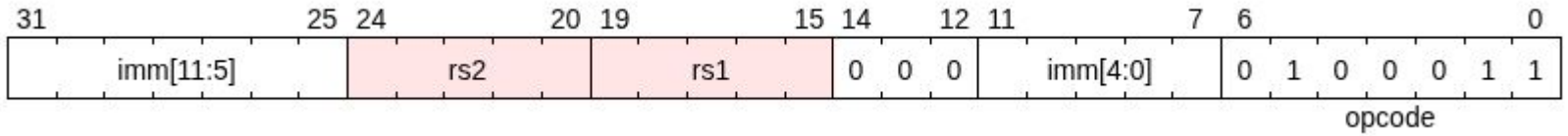
Tipos de Instucciones

Tipo S

- Son operaciones de Store



1.41. sb

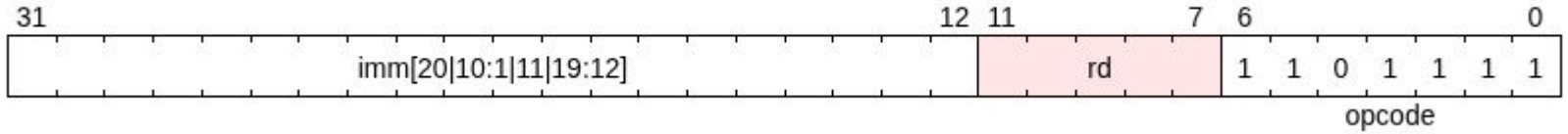


Tipos de Instrucciones

Tipo J

- Operaciones de salto incondicional
- La dirección a la que se salta es la almacenada en el registro «rd».

jal



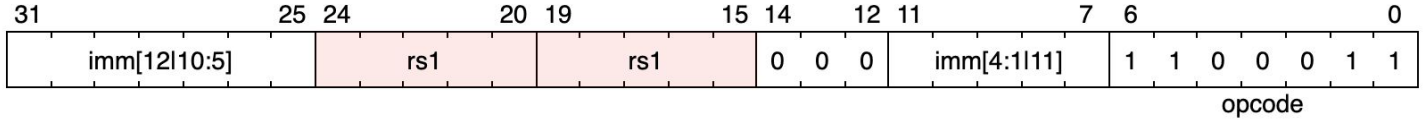
Tipos de Instucciones

B-Type (Ramificación Condicional)

- Saltan si se da un condición determinada.

1.46. beq

Encoding



Tipos de Instucciones

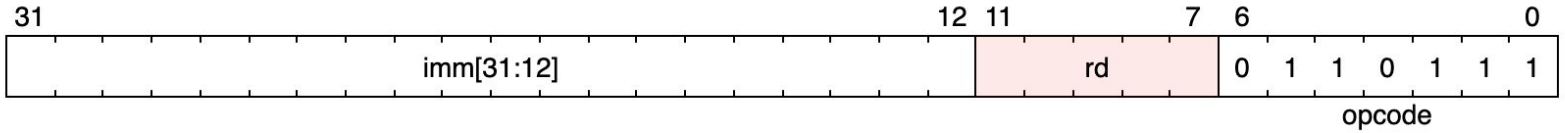
U-Type (Inmediato Superior)

- Se caracterizan por utilizar un inmediato de 20 bits para cargar la parte más significativa (superior) de un valor o una dirección.

1.1. lui

load upper immediate.

Encoding



Requerimientos

- El procesador debe ser capaz de programarse y reprogramarse a través de comandos de UART.
- El clock no debe verse intervenido en ninguna parte del proyecto
- Hay elementos que requieren de su ingenio y toma de decisiones, documenten estas decisiones y su porqué.
- Ser creativos al momento de mostrar los datos e interactuar con ellos (GUI, TUI, CLI).



Instrucciones

Instrucciones a implementar

- R-type (Registro a Registro)
  - add, sub, sll, srl, sra, and, or, xor, slt, sltu
- I-Type (Inmediato/Carga)
  - lb, lh, lw, lbu, lhu
  - addi, andi, ori, xori, slti, sltiu, slli, srli, srai
  - jalr
- J-Type (Salto Incondicional)
  - jal
- S-Type (Almacenamiento)
  - sb, sh, sw
- B-Type (Ramificación Condicional)
  - beq, bne
- U-Type (Inmediato Superior)
  - lui

Debug unit

- Se deben enviar a la PC a través de la uart:
  - El contenido de los 32 registros.
  - El contenido de los latches intermedios.
  - Contenido de la memoria de datos usada

Carga de Programa

El Programa debe:

- Estar escrito en ensamblador.
- Contar con un mecanismo para traducir instrucciones a lenguaje máquina para ser enviados al procesador.
- Contar con un instrucción HALT o instrucción de stop.

El Sistema debe:

- Permitir la programación del procesador utilizando el software escribiendo la memoria de programa
- Permitir la reprogramación del procesador de manera dinámica

- Responder a las preguntas:
  a. *Es necesario vaciar la memoria?*
  b. *Y los registros?*
  c. *Se necesita vaciar el pipeline?*
  d. *Y la memoria de programa?*

Importante: la carga del programa debe llevarse a cabo a través de la comunicación UART y sin resintetizar el procesador.

Modos de operación

- Debe permitir dos modos de operación:
  - Continuo, se envía un comando a la FPGA por la UART y esta inicia la ejecución del programa hasta llegar al final del mismo. Llegado ese punto se muestran todos los valores indicados en pantalla.
  - Paso a paso: Enviando un comando por la uart se ejecuta un ciclo de clock. Se debe mostrar a cada paso los valores indicados.

En ambos casos, es necesario que el pipeline quede vacío al momento de terminar la ejecución.

Responder: Qué sucede si en mi memoria no se encuentra una instrucción de parada?

Clock

Llegada la integración deben preguntarse.

- Cual es el camino crítico de mi sistema?
- Este camino crítico genera Skew en mi sistema? que consecuencias tiene esto?

De encontrarse Skew:

- Encontrar la frecuencia de funcionamiento optima de mi sistema.
- Generar métricas de funcionamiento utilizando las herramientas de Vivado.
- Aplicar la frecuencia de funcionamiento en mi sistema.

Tips

1. No copien, inspirence.
2. Investiguen herramientas que brinda Vivado (IPCores para las memorias, Clock Wizard).
3. Antes de escribir la primera línea:
    a. Disenien, dibujen y esquematizen
    b. Imaginen los modulos
        i. Piensen en cómo probarlos
        ii. Como interactuan entre si
        iii. Que herramientas que ya usaron o vieron pueden utilizar
    c. Como voy a interactuar con mi sistema?
    d. Como puedo interpretar el estado actual de mi sistema mientras lo uso?

Todo lo que se les pide, ya existe y ya se implementó 1000 veces.
Asegurense del que hagan ustedes sea único.

Es lo ultimo, disfrutenlo

Bibliografia

- **Instrucciones:**
  RISC-V instruction set
- **Pipeline:**
  Libro